

INSTITUTO PROFESIONAL
SAN SEBASTIAN

link repositorio privado en github:
https://github.com/calvareza/eva1_alvarez_laguna_barbaroy.git

Asignatura: Desarrollo Backend

Prof: Patricio Eduardo Silva Cartes

Integrantes:

Carlos Alaverez

Neervison Laguna

Agustina Barbaroy

Introducción

La presente entrega técnica detalla el desarrollo y la implementación para el proyecto de Core Bancario de **Fintech Solutions S.A.** El objetivo central de esta etapa ha sido la consolidación de una infraestructura moderna basada en microservicios, utilizando **Docker** como motor de orquestación.

En primer lugar, se ha establecido un entorno de contenedores que garantiza la independencia tecnológica y la portabilidad del sistema. **Posteriormente**, se ha trabajado en la capa de persistencia mediante **MySQL**, asegurando que la gestión de datos críticos —como la información de clientes— se realice bajo estándares de seguridad y eficiencia. Esta introducción marca el punto de partida de un sistema diseñado para ser escalable, robusto y alineada con las demandas actuales del sector financiero.

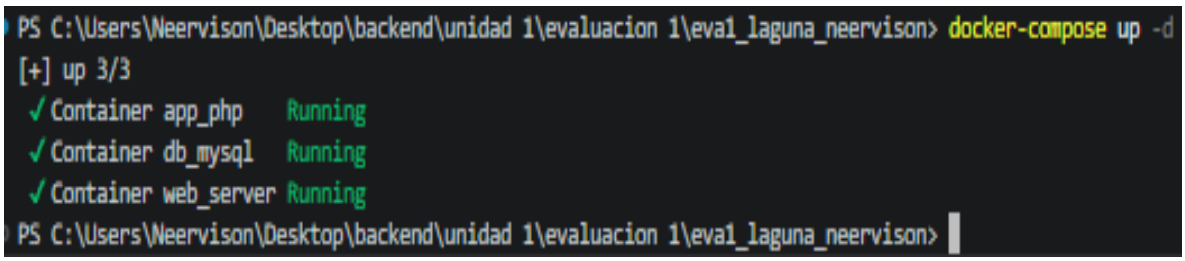
2. Arquitectura de Microservicios (Docker)

Se ha diseñado una infraestructura compuesta por tres servicios independientes que interactúan de forma orquestada:

- **Servidor Web (Nginx):** Configurado para gestionar las peticiones HTTP en el puerto 80.
- **Servidor de Aplicación (PHP-FPM 8.2):** Contenedor que procesa la lógica de Laravel y se encarga de la comunicación con la base de datos.
- **Servidor de Base de Datos (MySQL 8.0):** Almacena la información de forma persistente, utilizando volúmenes para asegurar que los datos no se pierdan al reiniciar los contenedores.

Evidencia de Infraestructura:

El archivo **docker-compose.yml** actúa como el director de orquesta, permitiendo que con un solo comando **(docker-compose up -d)** todo el sistema se levante de forma sincronizada.



Puesta en Marcha del Proyecto

Para que el sistema sea funcional, el proyecto atraviesa tres hitos críticos de configuración que aseguran la sincronización de sus componentes:

- **Fase de Estructuración (Laravel):** Se define la lógica de negocio, rutas y controladores en el directorio /app. Sin esta base de código, los servicios de Docker carecerían de instrucciones de ejecución.
- **Orquestación e Implementación (Docker Compose):** Mediante el comando docker-compose up -d, los servicios independientes se levantan y vinculan el código local con el entorno virtualizado. En este punto, el servidor web Nginx puede interpretar el código fuente a través del puerto 80.
- **Activación de la Capa de Datos (Migraciones):** Finalmente, para habilitar la persistencia, se ejecutan las migraciones de Laravel. Este proceso transforma las definiciones de código en tablas físicas dentro del motor MySQL, permitiendo que la plataforma comience a operar con información real

3. Estructura de Directorios y Organización

Como equipo, aplicamos una estructura de carpetas profesional para separar las responsabilidades del sistema:

● **/docker:** Archivos de configuración y Dockerfiles.

● **/app/Http/Controllers:** Controladores de la lógica de negocio.

● **/routes:** Definición de endpoints de la API.

● **/database/migrations:** Definiciones de la estructura de la base de datos.

Diagrama y Flujo de Comunicación

El sistema opera bajo un modelo de comunicación síncrona entre servicios. El flujo de una petición estándar sigue este orden:

1. Entrada: El cliente (Postman) realiza una petición HTTP que es recibida por Nginx en el puerto 80.

2. Procesamiento: Nginx actúa como proxy inverso y redirige la solicitud vía FastCGI al contenedor PHP-FPM, donde Laravel procesa la lógica.

3. Datos: Si la operación requiere persistencia, el contenedor de aplicación se comunica con el servicio MySQL mediante el puerto 3306.

4. Salida: El sistema devuelve una respuesta estructurada en formato JSON al cliente original.

[CLIENTE / POSTMAN] │

│ *(1) Envío de Petición HTTP (Puerto 80)* ▼

[SERVIDOR WEB (NGINX)] │

│ *(2) Proxy Inverso via FastCGI* ▼

[APLICACIÓN (PHP-FPM / LARAVEL)] │

├──────── *(3) Consulta/Registro SQL (Puerto 3306)* ────────┐

│ │

▼ ▼

[GENERACIÓN DE RESPUESTA JSON] **[BASE DE DATOS (MYSQL)]** │

│ *(4) Retorno de Código 200 OK* ▼

[CLIENTE / POSTMAN]}

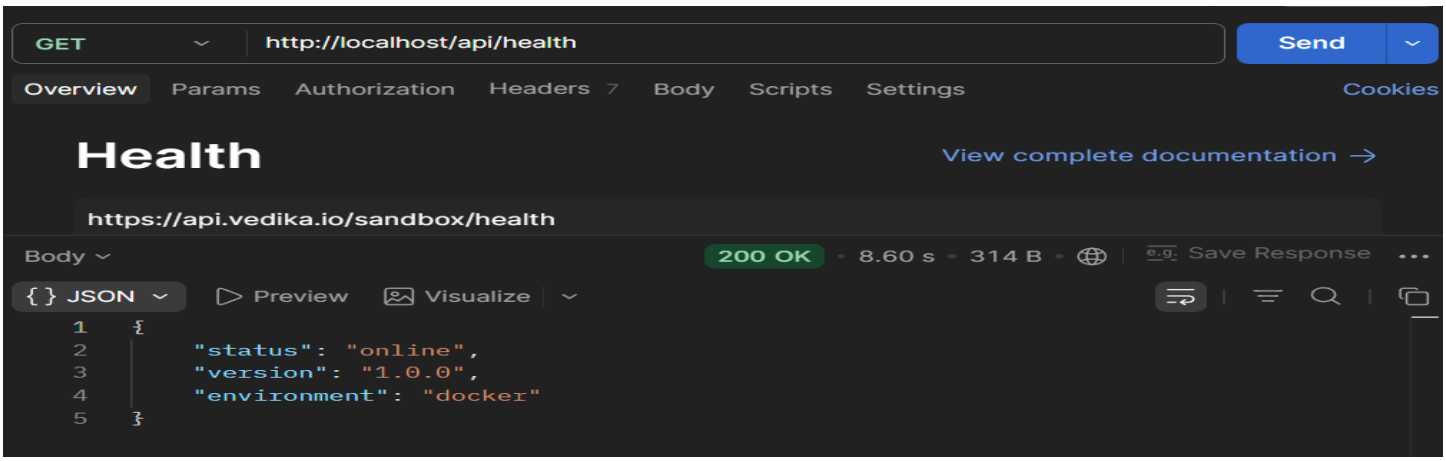
El diagrama representa la orquestación de microservicios mediante Docker Compose. Se destaca el uso de Nginx como proxy inverso para proteger el servidor de aplicación y el aislamiento de la base de datos MySQL, la cual solo es accesible internamente por el servicio PHP, garantizando así los estándares de seguridad requeridos por Fintech Solutions S.A.

4. Evidencias de Conectividad (Validación API)

Se validó la correcta comunicación entre el servidor Nginx y el contenedor PHP mediante una prueba de salud (Health Check).

Prueba en Postman:

- **Método:** GET
- **URL:** http://localhost/api/health
- **Respuesta Esperada:** {"status": "online"}



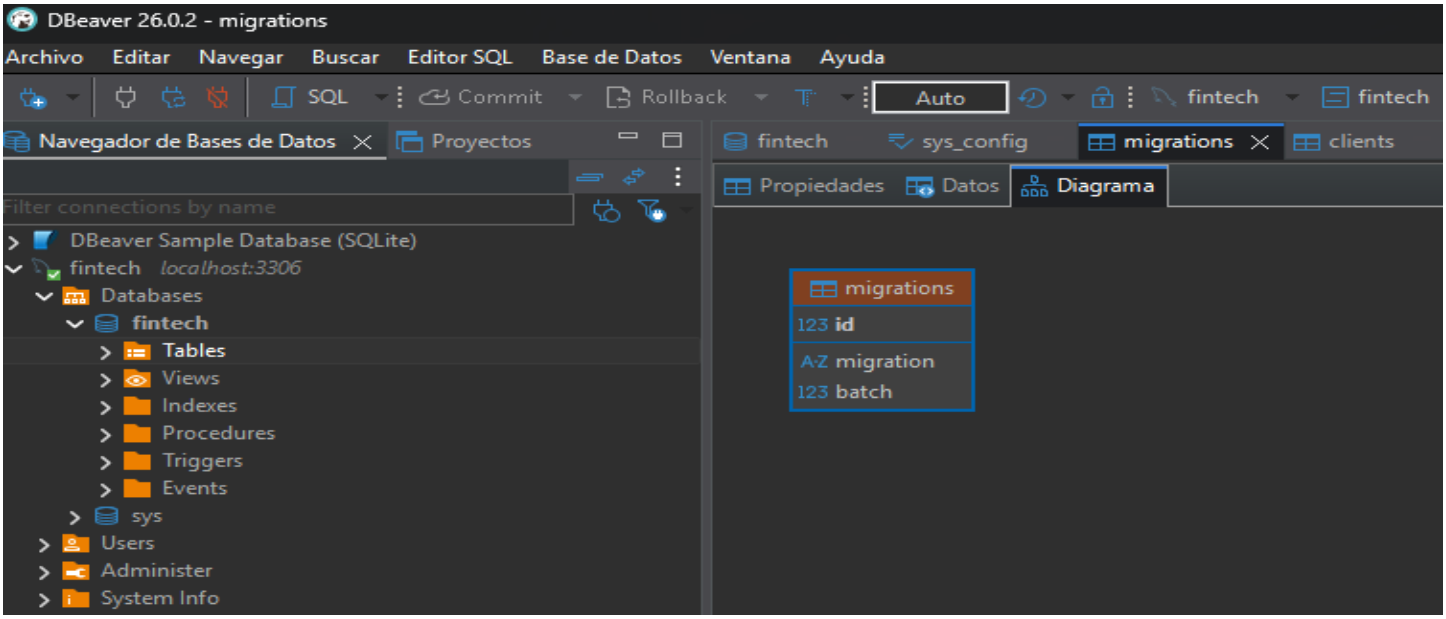
5: Gestión de Persistencia y Validación

Para garantizar la integridad y disponibilidad de la información, se implementó un flujo completo de persistencia utilizando **Laravel Migrations** y **Eloquent ORM**.

A. Creación de Estructura (Migraciones)

Se definió la tabla clients mediante una migración programada para automatizar la creación de esquemas en el motor MySQL.

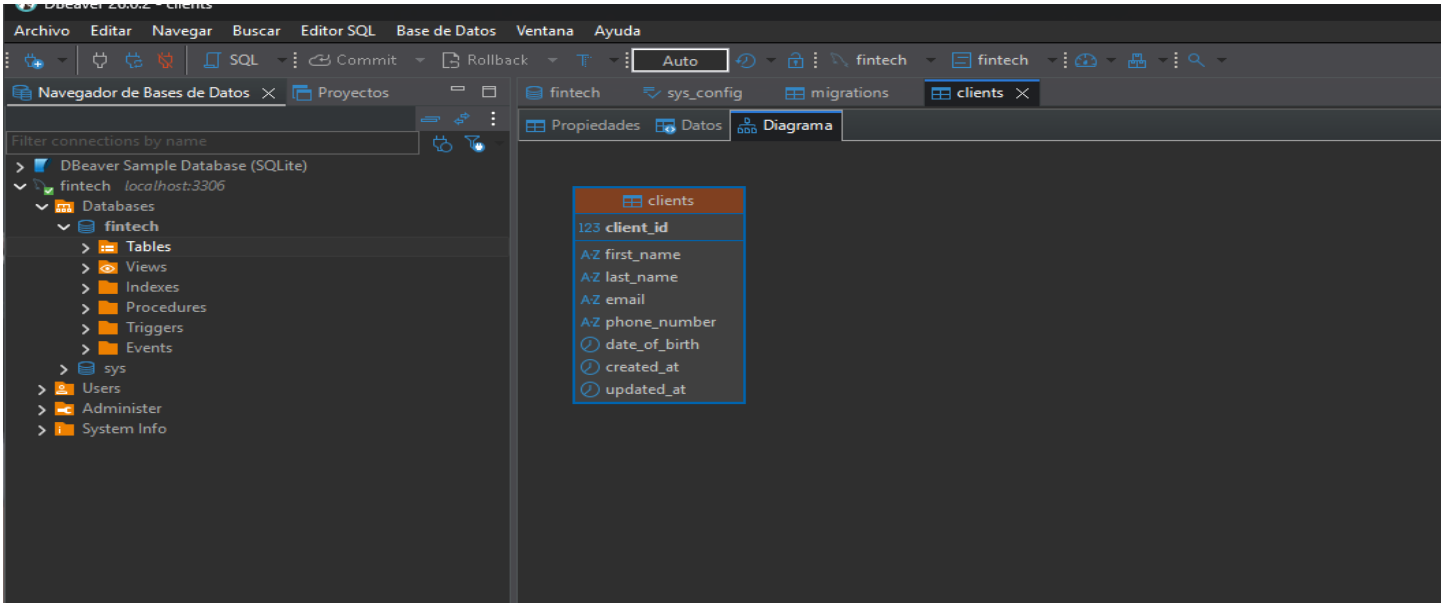
Evidencia: Archivo de migración generado en el directorio database/migrations.



B. Validación Visual del Modelo Relacional (DBeaver)

Utilizando la herramienta **DBeaver**, se verificó la correcta implementación de la tabla en el contenedor MySQL. Se confirma la existencia de la clave primaria client_id y los campos de auditoría necesarios para el seguimiento de registros.

Evidencia: Diagrama Entidad-Relación (ER) de la tabla clients.



C. Poblado de Datos y Prueba de Comunicación (Seeders)

Para validar que el backend tiene permisos de escritura y una conexión estable con la base de datos, se ejecutó el comando **docker-compose exec app_php php artisan migrate:fresh --seed**. El resultado exitoso (**DONE**) confirma que los datos de prueba fueron insertados correctamente a través del modelo Client.

Evidencia: Consola de comandos mostrando la migración y el seeder finalizados con éxito.



Conclusión

En conclusión, el desarrollo de esta primera etapa ha permitido consolidar un ecosistema de backend robusto y escalable para la Fintech. La implementación de **Docker** fue clave para neutralizar las inconsistencias de entorno, garantizando que la aplicación se ejecute de forma idéntica en cualquier estación de trabajo.

A través de **Laravel**, se logró una separación clara de responsabilidades: el uso de **Migraciones** otorgó un control de versiones riguroso sobre el esquema de la base de datos, mientras que los **Seeders** permitieron validar la persistencia y la lógica de negocio de manera inmediata. Este trabajo no solo cumple con los requisitos técnicos exigidos, sino que establece una base arquitectónica sólida, profesional y preparada para la integración de futuras funcionalidades bancarias complejas.